

Decentralized Proof of Personhood

A plain-language guide to everything we built, how it works, why we added a second (voice) liveness factor and an anti-Sybil face check, and how to defend every piece of it in front of a panel.

VTU B.E. Project · SKIT, 2025–26

Team 14 — Revanth, Shashank, Ajay, Chinmayee

This supersedes the earlier [Seminar-Explainer.pdf](#), written when the system had only one liveness factor (face/gesture) and no duplicate-identity check. Everything in that version is still accurate background reading; this document folds in what changed since: the voice challenge, the anti-Sybil face-duplicate check, the test suite, and hosting.

Contents

1. The 60-second pitch
2. What problem are we actually solving?
3. The big picture — three services, two liveness factors
4. Concepts you must understand
5. Why we added a second (voice) factor — the actual story
6. Why the anti-Sybil face check, instead of just letting wallets mint freely
7. Walking through the app, step by step
8. A tour of every file
9. What it can't do yet — know these cold
10. Ideas for future work
11. Anticipated questions and how to answer them
12. Where it's hosted
13. Glossary

1. The 60-second pitch

"Online, it's hard to prove you're a real, unique human without handing over your Aadhaar or your face to a company's server. Our project lets someone prove they're a unique human using only their own device — a face-tracking model runs entirely in their browser to check they performed a random physical action like blinking twice, and then they read a randomly chosen sentence out loud, transcribed live in the browser, as a second independent liveness factor. A blockchain then records that proof as a token in their crypto wallet that can never be sold, transferred, or duplicated — and critically, the system also recognizes if the same face tries to register a second token under a different wallet, and refuses. One human, one token, forever. No company holds their video, their photo, their audio, or their identity — the backend only ever sees a wallet address, a derived face descriptor, and a speech transcript."

2. What problem are we actually solving?

This category of problem is called **Sybil resistance** — stopping one person from pretending to be many people. **Proof of Personhood** is the general name for systems that answer "is this a unique real human?" without a government ID. Well-known examples are **Worldcoin / World ID** (iris-scanning hardware orb) and **Humanity Protocol**.

Our project is a lightweight, no-special-hardware version: a laptop/phone webcam plus a free on-device face model, now paired with a second factor — the device microphone plus live browser

speech transcription — and a check that the *same person* isn't quietly collecting more than one token.

3. The big picture — three services, two liveness factors

The project still has exactly three independently-run pieces of software — that part hasn't changed. What's new is that "verify" is no longer one check, it's a short pipeline of two independent liveness factors plus a duplicate-identity check, all before the backend will mint.

Piece	Technology	Its one job
Smart contract	Solidity, Polygon Amoy	Permanently record "this wallet is verified"; physically block every transfer after mint.
Backend	Python, FastAPI	Only party allowed to mint. Issues both challenges, re-scores the voice transcript itself, runs the face-duplicate check, pins metadata to IPFS, signs the mint transaction.
Frontend	React, running in the browser	Connect wallet, run face-tracking <i>and</i> live speech transcription, compute the face descriptor, show the final token.

UPDATED ANALOGY

It's still a nightclub with a wristband (the contract) and a bouncer (the backend). What's new: the bouncer now asks for two forms of ID instead of one — "do the thing" *and* "say the thing" — and keeps a (privacy-preserving) sketch of everyone's face so the same person can't come back through a side door with a different name and get a second wristband.

4. Concepts you must understand

Sections 4.1–4.9 (blockchain/wallet/testnet, smart contracts/gas, NFTs/ERC-721, soulbound tokens/EIP-5192, liveness detection, MediaPipe/blendshapes, IPFS/Pinata, FastAPI/Web3.py/ethers.js, `.env` files) are unchanged from the original explainer — read that document for those. Here are the concepts that are **new** in this version.

4.10 Voice liveness via challenge-response NEW

The same challenge-response idea used for "blink twice" extends naturally to speech: the backend picks one of 8 short, varied sentences at random (e.g. "*Blockchain technology enables secure and transparent digital identity.*") and the user must read that exact sentence aloud, right now. A pre-recorded clip of someone else's voice, or a generic answer, won't match the *specific, unpredictable* sentence that was just issued.

4.11 The Web Speech API NEW

Chrome (and other Chromium browsers) expose a built-in `SpeechRecognition` JavaScript API that transcribes microphone audio to text live, in the browser, with no library to install. We use it instead of running a speech-to-text model ourselves — it needs an internet connection and only works reliably in Chromium browsers, which is why the UI falls back to a manual "type what you said" box if it's unavailable, so a demo never gets stuck.

4.12 Transcript matching (and why the backend re-scores it) NEW

The frontend shows a live "match %" meter using a simple word-overlap formula (a [Dice coefficient](#), see glossary) purely so the user gets instant feedback while speaking. That number is *never trusted for the actual security decision* — when the transcript is submitted, the backend independently re-computes a similarity score (Python's `difflib.SequenceMatcher`, a character-level fuzzy-match ratio) between what was submitted and the exact sentence *it* issued, and only proceeds if that score is at least `AUDIO_MATCH_THRESHOLD` (0.65, i.e. 65% similar) — a classic "never trust the client" defensive pattern.

4.13 Geometric face descriptors & cosine similarity NEW

While the face-tracking model is already running for the gesture challenge, it produces 478 (x, y, z) landmark points per frame. We pick 10 stable ones (eye corners, nose tip, mouth corners, chin, forehead, cheeks, brow-bridge) and compute every pairwise distance between them — 45 numbers total — normalized by the distance between the two eye corners, so the vector doesn't change just because someone leaned closer to the camera. This is a **geometric descriptor**, a classic pre-deep-learning face-recognition technique: simple, fully explainable, and good enough to answer "is this roughly the same face as before?", though not as discriminating as a modern deep embedding model (see Section 9).

Two descriptors are compared with **cosine similarity** — a standard way to measure how similar two numeric vectors are regardless of their scale, producing a score from -1 to 1 (in practice 0 to 1 here). A score at or above `FACE_MATCH_THRESHOLD` (0.93) is treated as "the same face."

4.14 Sybil resistance, at the wallet level vs. the human level NEW

The original design only proved "this *wallet* passed a liveness check" — nothing stopped one human from doing that twice with two wallets. This version adds a second layer: an anti-Sybil check at the *human* level, using the face descriptor as a (imperfect, prototype-grade) stand-in for "the same person," independent of which wallet they're connected with.

5. Why we added a second (voice) factor — the actual story

This is worth being able to say in your own words, because it's exactly the kind of "why did you do that" question a panel asks:

- **Defense in depth.** Any single liveness signal has a plausible attack. A high-quality deepfake video or a screen replay can, in principle, fool blendshape-threshold gesture detection. Requiring a

second, independent channel — audio, with its own separate attack surface (you'd now need a deepfake that also perfectly lip-syncs an unpredictable sentence, live, in real time) — meaningfully raises the bar, even though neither factor alone is unbreakable.

- **It was explicitly in our own "future work" list.** The original Phase 1 report's literature survey centers on text-dependent / text-prompted speaker verification (RSR2015, RedDots — see the References document), and our own "future enhancements" section named stronger, multi-signal liveness as a direction. This is that direction, actually built.
- **It's cheap to add well.** Because the browser already exposes microphone access and (in Chrome) free live transcription, we could add a whole second modality with zero new paid services and zero heavy ML dependencies on our own backend — just `difflib`, which is in Python's standard library.

BE UPFRONT ABOUT THIS IF ASKED

We are *not* doing voice-biometric speaker identification (matching a voiceprint to a specific registered person) — we're only checking that the live microphone audio spells out the specific sentence we just issued. That proves "a live human spoke this, right now," not "this is unmistakably the same voice as last time." If pressed on it: "that's a correct observation — ours is a liveness check, not a speaker-recognition system; combining the two would be natural future work."

6. Why the anti-Sybil face check, instead of just letting wallets mint freely

We were directly asked, mid-project, "can a wallet mint more than once, or at least let me get my token again some other way?" The honest engineering answer we settled on:

- **Allowing repeat minting would be a regression, not a feature.** The single property this whole project exists to demonstrate is "one human, one token." A bypass for that — even a "just for testing" one — would directly contradict the thing being graded.
- **So we solved the *real* underlying need instead.** What was actually missing wasn't "let me mint twice," it was "stop a *different* wallet from quietly minting a second token for the same person" — which is the harder, more valuable half of Sybil resistance, and something the original design explicitly left as an open limitation (see the original explainer's Q&A: "*Nothing stops a person from owning multiple wallets and getting one token per wallet today*" — that sentence is no longer true in this version, within the limits of our lightweight face descriptor).
- **Rehearsal is still easy without weakening the real guarantee.** Switching MetaMask test wallets between rehearsal runs already works (each new wallet is a fresh on-chain slate); a dev-only, off-by-default endpoint (`ALLOW_DEMO_RESET=true` + `POST /dev/reset-face-registry`) clears just the local face registry for repeat rehearsals, without touching the blockchain and without existing anywhere near the real demo's default configuration.

7. Walking through the app, step by step

Steps 1–2 (connect wallet, check `isVerified`) and the mechanics of the gesture challenge itself are unchanged from the original explainer. Picking up from where the gesture challenge completes:

Step — gesture challenge completes

Instead of immediately calling the backend, the frontend now holds onto two things from that moment: which gesture challenge was completed, and the geometric face descriptor computed from the last clearly-visible frame (`frontend/src/lib/faceEmbedding.js`). The app then moves to a new "Voice" step instead of minting immediately.

Step — voice challenge

The frontend fetches a random sentence from `GET /challenge/audio`, the user reads it aloud, and (in Chrome) `SpeechRecognition` transcribes it live with a running match-percentage meter. The user hits "Submit & Mint."

Step — one combined verification request

Only now does the frontend call `POST /verify` — bundling the wallet address, which gesture challenge was done, the audio sentence id + transcript, and the face descriptor, all in one request. The backend, in order:

1. Checks `isVerified` on-chain (unchanged from before).
2. Re-scores the transcript against the sentence *it* issued; rejects (HTTP 400) below 65% match.
3. Compares the face descriptor against every descriptor already on file; if a *different* wallet's descriptor is $\geq 93\%$ similar, rejects (HTTP 409) with a clear message naming the already-registered wallet.
4. Only if both checks pass: pins metadata to IPFS, mints the Soulbound Token, and registers this descriptor against this wallet for future checks.

8. A tour of every file

Unchanged files (`SoulboundToken.sol`, `deploy.js`, `hardhat.config.js`, `WalletConnect.jsx`, `VerificationBadge.jsx`) are covered in the original explainer. New/changed files:

File	What it is
<code>backend/main.py</code>	Now five endpoints instead of three: health check, <code>GET /challenge</code> (visual), <code>GET /challenge/audio</code> (voice), <code>POST /verify</code> (re-scores audio, runs the face-dedup check, mints), plus two dev/rehearsal-only

	endpoints behind <code>ALLOW_DEMO_RESET</code> .
<code>backend/tests/test_main.py</code>	21 pytest cases covering the text-similarity scorer, the cosine-similarity scorer, the full <code>/verify</code> flow, the anti-Sybil block/allow cases, and the dev endpoints — all run against a temporary SQLite file with real network/blockchain calls forced off.
<code>backend/pop_registry.db</code>	The local face-descriptor registry (SQLite, one row per wallet). Runtime data only — git-ignored, never committed, contains no images.
<code>frontend/src/components/AudioChallengeRecorder.jsx</code>	The new voice challenge screen: fetches the sentence, drives <code>SpeechRecognition</code> , shows the live match meter, falls back to manual text entry, and is the component that actually calls <code>POST /verify</code> (moved here from the gesture component, since voice is now the last step before minting).
<code>frontend/src/lib/faceEmbedding.js</code>	Turns MediaPipe's 478 landmarks into the 45-number geometric descriptor described in Section 4.13.
<code>frontend/src/lib/textSimilarity.js</code>	The client-side "match %" meter only — a word-overlap (Dice) score, cosmetic/UX only, never the security decision.
<code>render.yaml</code> , <code>backend/runtime.txt</code> , <code>DEPLOYMENT.md</code>	Free hosting: a Render "Blueprint" for the backend and a step-by-step guide for both services (Section 12).

9. What it can't do yet — know these cold

Everything in the original explainer's limitations section still applies (deepfake/screen-replay risk, centralized minter trust, no rate-limiting). What's new or changed:

- **The face descriptor is a geometric sketch, not a deep embedding.** It's explainable and cheap, but less discriminating than a trained model like FaceNet/ArcFace — similar-looking relatives, identical twins, or unusual lighting/pose could plausibly cross the 93% threshold incorrectly in either direction. This is a deliberate prototype-scope trade-off, named as such.

- **Voice liveness proves "a human spoke this sentence now," not "this is the same voice as last time."** It is not speaker-recognition (Section 5's "be upfront" box).
- **Web Speech API needs Chrome and an internet connection.** Other browsers, or a venue with no wifi, fall back to a lower-assurance manual-typing path.
- **The free-tier host for the backend doesn't guarantee permanent disk persistence.** See Section 12 — the face registry can be reset by a Render cold-start cycle on the free plan.
- **Still no rate-limiting or signed attestations** — unchanged limitations from the original design; see Section 10.

10. Ideas for future work

Carried over from the original explainer (signed attestations, deeper anti-spoofing, mainnet migration, rate limiting, contract test suite, a real-world use-case demo), plus new ones specific to this version:

1. **Swap the geometric face descriptor for a deep embedding model** (FaceNet/ArcFace, run on-device via ONNX/TensorFlow.js) for materially stronger duplicate-detection accuracy.
2. **Add real voice-biometric speaker verification** on top of the current liveness-only check, so the system can also answer "is this specifically the same registered voice."
3. **Offline speech-to-text** (e.g. a small on-device Vosk model) to remove the Chrome + internet dependency for the voice challenge.
4. **Move the face registry to a real hosted database** (Postgres) so it survives host cold-starts, instead of a single SQLite file.

11. Anticipated questions and how to answer them

Q: Why two liveness factors instead of one?

A: Defense in depth — a convincing attack on gesture-based face liveness (e.g. a very good deepfake) is a different kind of attack than one that also needs to produce a live, correctly-lip-synced reading of an unpredictable sentence. Neither is unbreakable alone; combined, the bar is meaningfully higher.

Q: Is the voice check actually verifying *who* is speaking?

A: No — it verifies *that a live human* is speaking the specific sentence just issued. It's a liveness check, not speaker-recognition. We say this plainly; see Section 5.

Q: What stops someone from using a second wallet to get a second token?

A: This is the one answer that changed from our last documentation. Previously: nothing did. Now: the backend compares a geometric descriptor of the face against every face already on file: if a new verification attempt's face is $\geq 93\%$ similar to one already registered under a *different* wallet, the mint is rejected with a 409 error naming the existing wallet. It isn't perfect — it's a lightweight geometric descriptor,

not a production face-recognition model — but it directly addresses the gap, and the threshold/behavior is fully configurable (`ENABLE_FACE_DEDUP` , `FACE_MATCH_THRESHOLD`).

Q: Where is my face or voice data stored?

A: Nowhere, as raw data. Camera and microphone processing both happen entirely on-device. The backend only ever receives: a wallet address, which gesture challenge was completed, a speech transcript (text, not audio), and a 45-number geometric descriptor (not an image). None of those can be turned back into a photo or an audio recording.

Q: What if the face-match threshold produces a false positive — two different people incorrectly flagged as the same?

A: That's an honest, named limitation of using a lightweight geometric descriptor instead of a deep embedding model (Section 9) — it's a deliberate prototype-scope trade-off, and the threshold is a tunable config value specifically so it can be calibrated against real test data rather than hard-coded.

Q: Why not trust the client's own "match: 92%" score instead of re-checking on the server?

A: Because the client is exactly what an attacker controls. The number shown while speaking is only a UX convenience; the actual pass/fail decision is always recomputed server-side against the sentence the server itself issued, using the server's own copy of the correct answer.

Q: How did you test this?

A: 21 automated pytest cases covering the scoring math (text similarity, cosine similarity), the full `/verify` flow (success and failure paths), the anti-Sybil block/allow behavior specifically, and the dev/rehearsal endpoints — run against a temporary database with all real network and blockchain calls forced off, so the suite is fast, repeatable, and never touches real funds or a real Pinata account.

Q: Where does this run — is it hosted anywhere?

A: The frontend is deployable to Vercel's free tier; the backend to Render's free tier. See Section 12 for the exact setup and its one honest caveat (free-tier disk persistence).

(All questions from the original explainer's Section 10 — photo spoofing, where face data is stored, what stops transfers, why Polygon, centralization of the minter, gas — still apply unchanged and remain good answers.)

12. Where it's hosted

Two free services, connected to the same GitHub repo so both redeploy automatically on every push:

Piece	Host	Why that host
Frontend	Vercel (free)	Static Vite build + global CDN — exactly what Vercel is built for, zero config.

Backend	Render (free)	A real, persistently-running server — required because the face registry is a local SQLite file, which needs a machine that stays up. Serverless platforms like Vercel's Python functions don't guarantee that between invocations.
---------	---------------	---

BE UPFRONT ABOUT THIS IF ASKED

Render's free web service spins down after ~15 minutes idle and cold-starts (30–50s) on the next request. The face registry's disk is not guaranteed to survive that cycle on the free plan — fine within one continuous demo session, not a guarantee across days without upgrading to a paid persistent disk or an external database. Full steps and this exact caveat are written out in [DEPLOYMENT.md](#) at the repo root.

13. Glossary

Terms already defined in the original explainer (blockchain, wallet, testnet, smart contract, gas, NFT, ERC-721, Soulbound Token, EIP-5192, Sybil resistance, Proof of Personhood, liveness detection, challenge-response, MediaPipe, blendshape, on-device inference, IPFS, Pinata, FastAPI, Web3.py/ethers.js, MetaMask, environment variable, RPC) aren't repeated here. New terms:

Term	Meaning
Web Speech API	A browser-native JavaScript API (best supported in Chrome) that transcribes live microphone audio to text without any server or library.
Geometric face descriptor	A fixed-length list of numbers describing a face purely as distances between a handful of landmark points — simple and explainable, as opposed to a trained deep-learning embedding.
Dice coefficient	A simple word-overlap formula for comparing two pieces of text; used here only for the live, cosmetic "match %" meter.
<code>difflib.SequenceMatcher</code>	A character-level fuzzy text-matching tool built into Python's standard library; used server-side for the authoritative voice-match score.
Cosine similarity	A standard measure of how similar two numeric vectors are, regardless of their overall scale/magnitude — used here to compare two face descriptors.
Hermetic test	An automated test that never depends on real external services (network, blockchain, third-party APIs) — ours force IPFS/Pinata and blockchain calls off so the suite is fast and repeatable.
pytest	The Python testing framework used for the backend's 21 automated tests.
CORS (Cross-Origin Resource Sharing)	A browser security rule that blocks a web page from calling an API on a different domain unless that API explicitly allows it — why the backend keeps an <code>ALLOWED_ORIGINS</code> list.

Vercel / Render	Hosting platforms used for the frontend and backend respectively (Section 12); both have free tiers used here.
Idempotent	An operation that produces the same end result no matter how many times it's repeated — re-verifying the same wallet with the same face is treated this way (no error, no duplicate).

Prepared as a study aid for the seminar defense — an update to the original Seminar Explainer. Read Sections 1, 5, 6, and 11 the night before; those carry the most weight for "why did you add this" and "what stops X" questions in a live Q&A.